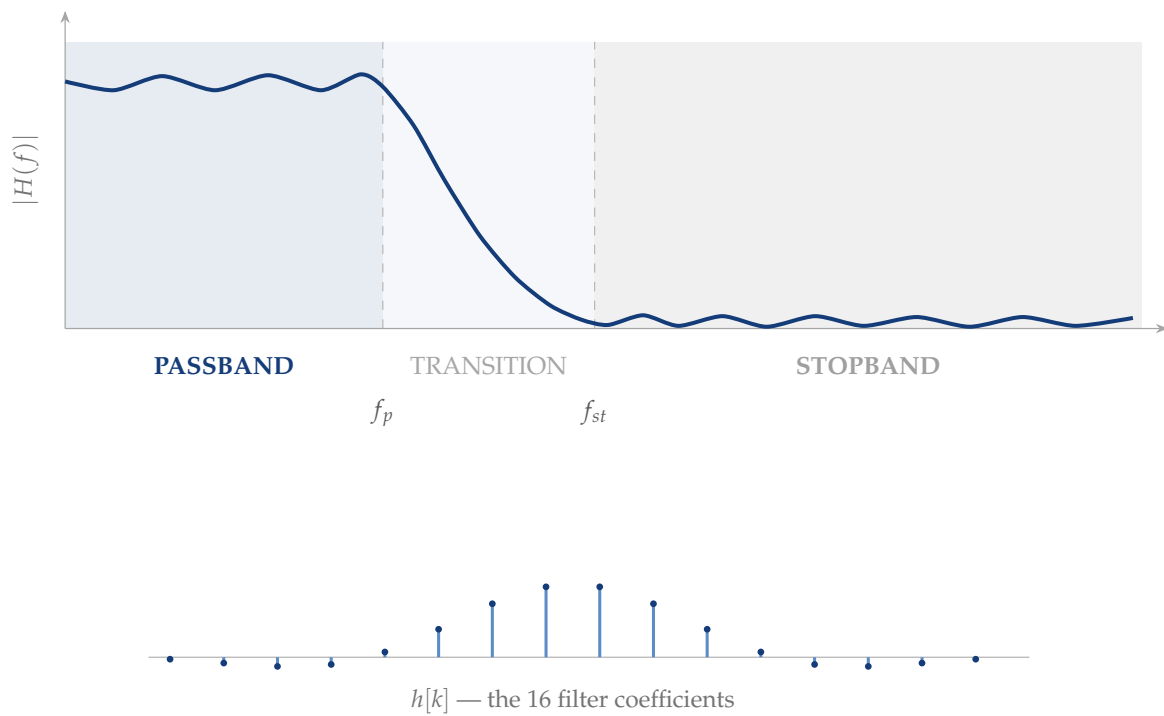

Direct-Form FIR Filter Mini-Project

From Specification to RTL

ZDC Workshop



Deliverables due: One week from project announcement

1 Project Overview

In this mini-project you will take a digital filter from a written specification all the way down to synthesisable RTL, and then prove that your hardware behaves the way the mathematics said it would.

The flow will be as follows:

1. Read the filter specification (sampling rate, band edges, ripple, attenuation).
2. Design the filter in MATLAB's `FilterDesigner` and obtain the coefficients.
3. Quantise the coefficients to fixed-point integers.
4. Implement the filter in Verilog as a **direct-form** structure.
5. Generate quantised test signals in MATLAB and feed them to your RTL.
6. Compare the RTL output against the MATLAB **golden model**.

2 History and Motivation for FIR Filters

Filters have been a cornerstone of signal processing since the early days of analog electronics, where discrete components like resistors, capacitors, and inductors were carefully tuned to separate communication channels or shape radio frequencies. However, analog filters are highly susceptible to component tolerances, temperature drift, and aging.

As technology advanced into the digital era during the 1970s and 1980s, the need for precise, repeatable, and stable systems gave rise to digital signal processing. Digital filters perform mathematical operations on sampled discrete-time signals, completely removing the physical vulnerabilities of analog components.

We specifically use **FIR (Finite Impulse Response)** filters in digital IC design for two primary reasons:

- **Unconditional Stability:** FIR filters rely entirely on feedforward paths. Because they lack any feedback loops, they can never become unstable or oscillate out of control.
- **Linear Phase:** By making the coefficients symmetric, an FIR filter achieves strictly linear phase. This means all frequency components of the signal are delayed by the exact same amount of time. It preserves the time-domain wave shape without any phase distortion, a crucial requirement in telecommunications, audio processing, and digital data transmission.

3 Crash Course: Digital Filters

3.1 What a filter does

A digital filter takes a sequence of input samples $x[n]$ and produces a sequence of output samples $y[n]$, shaping the frequency content of the signal on the way through. A **low-pass filter** (LPF) lets low frequencies pass and attenuates high frequencies.

We are concerned with three certain regions:

- **Passband:** frequencies you want to keep, ideally with gain ≈ 1 .
- **Stopband:** frequencies you want to remove, with gain as close to 0 as possible.
- **Transition band:** the region in between. It cannot be zero width; the narrower you make it, the more taps your filter needs.

3.2 The FIR difference equation

An N -tap FIR filter is defined by

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n-k] = h[0] x[n] + h[1] x[n-1] + \cdots + h[N-1] x[n-N+1] \quad (1)$$

where $h[k]$ are the **filter coefficients** (the impulse response) and N is the **number of taps**. As the equation suggests, we will need the coefficients and a way to delay, multiply, and sum the terms. The coefficients come from MATLAB, and the hardware is nothing more than a literal implementation of that sum.

3.3 Direct-form structure

Equation (1) maps onto hardware as a chain of registers (the delay line), one multiplier per tap, and an adder tree that sums all the products. This is the **direct form**.

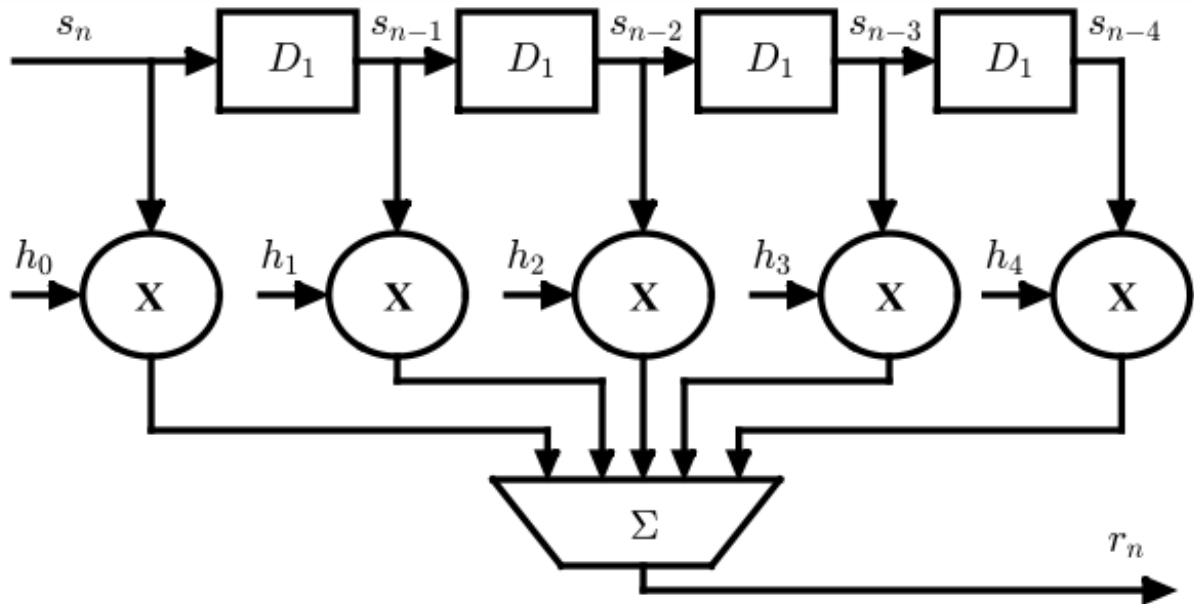


Figure 1: Direct-form realisation of an N -tap FIR filter.

Read the structure alongside Equation (1) until the correspondence is obvious: each register in the chain holds one $x[n-k]$, each multiplier applies one $h[k]$, and the adder tree performs the summation.

4 Filter Specification

To build an effective FIR filter, we must define its operational boundaries through a rigorous set of specifications. The sampling rate (f_s) dictates the maximum frequency we can process without aliasing (the Nyquist limit). The passband and stopband edges (f_p and f_{st}) define exactly where the filter allows signals to pass unaltered and where it begins to aggressively suppress them. The passband ripple determines the maximum allowed amplitude distortion for our wanted signals, while the stopband attenuation ensures interfering noise is buried deeply enough not to affect downstream processing.

Design a low-pass FIR filter meeting the following specification.

Table 1: Filter specification.

Parameter	Value	Notes
Filter type	Low-pass FIR	Equiripple (Parks–McClellan)
Sampling rate f_s	48 kHz	Nyquist = 24 kHz
Passband edge f_p	4 kHz	Signals below this must pass
Stopband edge f_{st}	10 kHz	Signals above this must be rejected
Passband ripple	≤ 1 dB	Peak-to-peak in the passband
Stopband attenuation	≥ 50 dB	
Number of taps N	16	Fixed — this is a hardware constraint
Filter order	15	Order = $N - 1$

5 Step 1: Designing the Filter in MATLAB

5.1 Using Filter Designer

Open the tool from the MATLAB command window:

```
filterDesigner
```

Then configure it as follows:

1. **Response Type:** Lowpass.
2. **Design Method:** FIR $\rightarrow$ Equiripple.
3. **Filter Order:** select *Specify order* and enter 15.
4. **Frequency Specifications:** Units = Hz, $F_s = 48000$, $F_{pass} = 4000$, $F_{stop} = 10000$.
5. **Magnitude Specifications:** use the weights to trade passband ripple against stopband attenuation. Increasing W_{stop} relative to W_{pass} pushes the stopband down at the cost of more passband ripple. Keep passband ripple within 1 dB.
6. Click **Design Filter**.

5.2 Inspecting the design

Use the analysis buttons in the toolbar to view, and **screenshot**, each of the following:

- **Magnitude response**
- **Phase response**
- **Pole/Zero plot**
- **Filter Information**
- **Filter Coefficients:** the $h[k]$ values.

5.3 Exporting the coefficients

Export the coefficients to the MATLAB workspace (*File $\rightarrow$ Export $\rightarrow$ Export to Workspace*)

6 Step 2: Fixed-Point Quantisation

Hardware cannot store real numbers. Every coefficient and every sample must become an integer with an agreed-upon binary point. We use the **Q1.15** format: a 16-bit signed two's-complement number with 1 sign bit and 15 fractional bits, representing values in the range $[-1, +1)$ with a resolution of 2^{-15} .

6.1 Quantising the coefficients

$$h_{int}[k] = \text{round}(h[k] \times 2^{15}) \quad (2)$$

```
h_int = round(h * 2^15);           % Q1.15 integer coefficients
h_q    = h_int / 2^15;             % quantised coefficients, back in real terms

% Always check what quantisation cost you:
fvtool(h, 1, h_q, 1);             % ideal vs. quantised magnitude response
```

Check for saturation

Verify that every value of `h_int` lies within $[-32768, +32767]$. If any coefficient has magnitude ≥ 1.0 , Q1.15 cannot represent it and you must rescale. For this filter all coefficients are comfortably below 0.3, so no rescaling is needed.

6.2 Writing values as hex files

All three quantised datasets you will need in Verilog — the coefficients, the input stimulus, and the golden-model output — are written using the same convention: **one 4-digit hexadecimal value per line, two's complement**, which `$readmemh` reads directly. Use one small helper for all of them:

```
function write_hex(filename, q_int)
    % Write a vector of Q1.15 integers as one 4-digit hex value per line,
    % two's-complement, ready for $readmemh in Verilog.
    fid = fopen(filename, 'w');
    for k = 1:length(q_int)
        fprintf(fid, '%s\n', dec2hex(typecast(int16(q_int(k)), 'uint16'), 4));
    end
    fclose(fid);
end
```

Coefficients are still hardcoded

The RTL itself must still hardcode the coefficients as `localparam` values (Step 3) — this project does not load coefficients from a file at runtime. Exporting `fir_coeffs.hex` with `write_hex('fir_coeffs.hex', h_int)` is still worth doing: it gives you a plain-text record you can diff your `localparam` values against, so a typo in the hardcoded coefficients is easy to catch.

6.3 Generating and quantising the test signals

Signals are quantised the same way. When generating your test signals in MATLAB, scale them to Q1.15 and write the samples out with `write_hex` (e.g. `write_hex('stimulus_case1.hex', x_int)`) so your testbench can load each stimulus file with `$readmemh`.

Why amplitude 0.4 and not 1.0?

The sum of the absolute values of the coefficients is approximately 1.31. If your input reaches full scale, intermediate sums inside the filter can exceed the $[-1, +1)$ range of Q1.15 and wrap around, producing spectacular garbage at the output. Keeping each of the two tones at amplitude 0.4 leaves comfortable headroom.

7 Step 3: RTL Implementation

7.1 Required structure

Implement the filter in **direct form**, exactly as written in Equation (1) and drawn in Figure 1:

- A shift register of 16 stages holding $x[n], x[n-1], \dots, x[n-15]$.
- 16 multiplications, one per tap.
- A combinational **adder tree** that sums all 16 products within the same clock cycle — *not* a serial multiply-accumulate spread over several cycles. This is what makes the one-sample-in, one-sample-out-per-clock interface below possible.

7.2 Coefficient storage

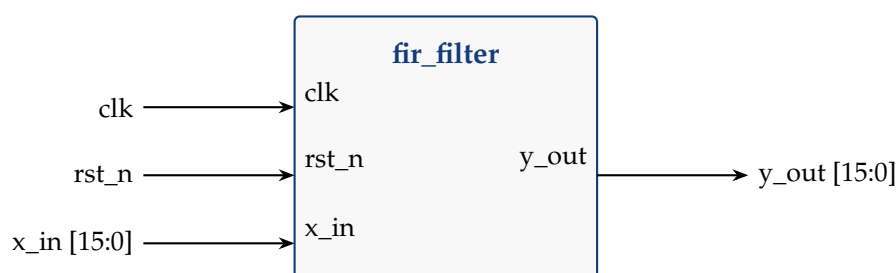
Coefficients must be **hardcoded** in the RTL using `localparam` — no external files, no memory initialisation:

```
1 localparam signed [15:0] H0 = 16'hXXXX; // h[0]
2 localparam signed [15:0] H1 = 16'hXXXX; // h[1]
3 // ... through H15
```

An array-style declaration is equally acceptable if you prefer it.

7.3 Interface

The input/output structure is illustrated in the interface block diagram below.



Use the following interface. It is deliberately minimal: one input sample is consumed on every clock edge and one output sample is produced on every clock edge.

On reset, clear the delay line and the output register.

7.4 Arithmetic and bit widths

This is the part most people get wrong. Work through the reasoning rather than copying the numbers.

The four extra accumulator bits are **guard bits**. Summing 16 products can grow the magnitude by up to a factor of 16, which is 2^4 , so four extra bits guarantee that the accumulation itself can never overflow.

Table 2: Required bit widths.

Signal	Width	Format	Reasoning
Input sample $x[n]$	16 bits	Q1.15	Given by the specification
Coefficient $h[k]$	16 bits	Q1.15	Given by the specification
Product $h[k]x[n-k]$	32 bits	Q2.30	$16 + 16$ bits
Accumulator	36 bits	Q6.30	$32 + \lceil \log_2 16 \rceil = 32 + 4$
Output $y[n]$	16 bits	Q1.15	Truncated from the accumulator

To bring the Q6.30 accumulator back to a Q1.15 output, keep bits 30 down to 15:

```

1 reg signed [35:0] acc;
2 // ...
3 y_out <= acc[30:15];      // truncate Q6.30 -> Q1.15

```

Signed arithmetic

Every declaration involved in the arithmetic must be signed, and every literal must be written in signed form (16'sh...). A single unsigned signal in the datapath will silently make the whole expression unsigned, and your negative samples will become large positive ones. This is the single most common bug in this project.

8 Step 4: Testbench and Test Cases

Write a Verilog testbench that reads the quantised stimulus **hex files** with \$readmemh, drives one sample per clock into the DUT, and captures the output.

You must exercise **three** cases. In all three, the wanted signal is a sine wave inside the passband and the interferer is a second sine wave. The expected attenuation values below come from the magnitude response — read them off your own design and check that your hardware agrees.

Case 3 is not a formality. A filter that attenuates everything would pass cases 1 and 2 and fail case 3, so this is the case that proves your filter is actually a filter and not just an attenuator.

Table 3: Required test cases.

#	Signal	Interferer
1	1 kHz	18 kHz
2	1 kHz	7 kHz
3	1 kHz	3 kHz

9 Step 5: Verification Against the Golden Model

Your RTL output must be compared against the MATLAB golden model (y_{ref} , computed with the *quantised* coefficients).

Export the MATLAB golden-model output to a hex file with write_hex, read it into the testbench with \$readmemh alongside the stimulus, and compare sample-by-sample in Verilog. Display the two signals together in the waveform viewer using analogue format.

To view a filtered sine wave meaningfully in ModelSim, set the radix of x_{in} and y_{out} to **decimal (signed)** and the waveform format to **analog**. Viewing them in hexadecimal tells you

nothing about the waveform shape — the hex files are just the storage/transfer format between MATLAB and Verilog, not how you should read the values on screen.

10 Deliverables

Submit a single archive named `FIR_<your group number>.zip` containing:

1. Verilog design codes:

- `fir_filter.v` — the RTL design, direct form, localparam coefficients.
- `fir_filter_tb.v` — the testbench covering all three cases.

2. FilterDesigner screenshots

- The specification panel showing your entered parameters
- Filter Information
- Magnitude (frequency) response, with markers on the band edges
- Phase response
- Pole/zero plot
- Coefficient listing

3. Waveform screenshots: one per test case, showing input and output together in analogue format with signed decimal radix.

4. Verification evidence: the comparison against the golden model including the plots or the mismatch report.

5. MATLAB scripts and hex files used to generate the coefficients (`fir_coeffs.hex`), stimulus (`stimulus_case*.hex`), and golden-model output (`golden_case*.hex`).

11 Common Pitfalls

- Forgetting signed somewhere in the datapath.
- Truncating from the wrong accumulator bits.
- Comparing the RTL output against the *input* instead of the golden model.
- Driving the delay line with blocking assignments (`=`) instead of non-blocking (`<=`) inside the clocked block.
- Letting the test signal amplitude reach full scale and overflowing the accumulator.
- Reading the modelsim waveform in hexadecimal and concluding nothing is happening.

Ask questions early. A filter that is not working for a reason you can explain is worth more than one that works for reasons you cannot.